As well as the original division into chunks, the addition of the partial sum from the previous block can also be parallelized. If the last element of each block is updated first, the remaining elements in a block can be updated by one thread while a second thread updates the next block, and so forth. This works well when there are many more elements in the list than processing cores, because each core has a reasonable number of elements to process at each stage.

If you have a lot of processing cores (as many or more than the number of elements), this doesn't work so well. If you divide the work among the processors, you end up working in pairs of elements at the first step. Under these conditions, this forward propagation of results means that many processors are left waiting, so you need to find some work for them to do. You can then take a different approach to the problem. Rather than doing the full forward propagation of the sums from one chunk to the next, you do a partial propagation: first sum adjacent elements as before, but then add those sums to those two elements away, then add the next set of results to the results from four elements away, and so forth. If you start with the same initial nine elements, you get 1, 3, 5, 7, 9, 11, 13, 15, 17 after the first round, which gives you the final results for the first two elements. After the second you then have 1, 3, 6, 10, 14, 18, 22, 26, 30, which is correct for the first four elements. After round three you have 1, 3, 6, 10, 15, 21, 28, 36, 44, which is correct for the first eight elements, and finally after round four you have 1, 3, 6, 10, 15, 21, 28, 36, 45, which is the final answer. Although there are more total steps than in the first approach, there's greater scope for parallelism if you have many processors; each processor can update one entry with each step.

Overall, the second approach takes $\log_2(N)$ steps of around $N$ operations (one per processor), where $N$ is the number of elements in the list. This compares to the first algorithm where each thread has to perform $N/k$ operations for the initial partial sum of the chunk allocated to it and then further $N/k$ operations to do the forward propagation, where $k$ is the number of threads. Thus the first approach is $O(N)$, whereas the second is $O(N \log(N))$ in terms of total number of operations. However, if you have as many processors as list elements, the second approach requires only $\log(N)$ operations *per processor*, whereas the first essentially serializes the